Table 1: Properties of the generated input files.

File#	Size	Lines	Functions	AST Nodes
1	5.1 KB	121	5	1 211
2	11 KB	209	9	3 211
3	99 KB	1 992	80	26 946
4	500 KB	10 328	412	134 564
5	1.1 MB	21 939	895	289 082
6	9.8 MB	204 649	8387	2 675 153
7	47 MB	971 014	40112	12 872 847

Additionally, based on the lifetime mask a table is created that contains the number of instructions to be generated for each intermediate instruction including spill instructions. Subsequently, instructions are removed according to the instruction mask and spill instructions are inserted. These operations are combined for the sake of performance. New instruction locations are computed and all generated instructions are copied to the final locations in the instruction buffer using a parallel scatter. Afterwards, any required spill instructions are inserted obtaining the stack offsets from the stack offset table. With all instructions in place, the virtual register numbers in each instruction are resolved to physical registers using the virtual register table. As the final step, the function offset table is recomputed to account for the removed and inserted instructions, and the target addresses for all jump instructions are corrected. The separate buffers containing per-instruction information are merged to construct the final instruction buffer.

4 EVALUATION

For the purpose of this feasibility study, we are interested in the effectiveness of the parallel algorithms and an indication of the performance of the parallel compiler compared to traditional CPU-based compilation. To investigate the effectiveness, we have performed a number of experiments to determine the scaling characteristics under increasing input sizes. Given that a custom programming language is used in this study, there is no source code available that represents real-world use cases and translation of real-world source code to this language is complicated. Therefore, we have randomly generated seven source code files that we believe will yield results indicative of the performance on real-world source code. The properties of these files are shown in Table 1, all files are syntactically and semantically correct.

The experiments were conducted on an NVIDIA GeForce RTX3090 GPU, hosted in a machine equipped with dual Intel Xeon Silver 4214R processors. The software configuration consists of CentOS 7, Linux kernel 5.4.126, Futhark 0.20, CUDA 11.2 and gcc 10.2.0 in release mode (`-O3`). All runtime measurements are obtained using the C++ `chrono::high_resolution_clock`. Care is taken that GPU and CPU work is synchronized before a timestamp is collected. To reduce noise, we repeat every individual experiment 30 times.

Figure 4a depicts a breakdown of the runtime from parsing up to the creation of the AST. Almost all stages show similar characteristics. Initially, the scaling appears flat, as a relatively large amount of time is spent launching the kernels and combining the results of the individual shader processors, making the actual processing time almost negligible compared to the overhead. From the third to the sixth input file we see a sub-linear scaling, for example for Semantic Analysis we observe a factor 6.4 increase in runtime for a 100x increase in input size. From the sixth input file onwards this nears linear scaling, with a 4.7x increase in runtime for a 4.8x increase in input size.

The results for the code generation stages are shown in Figure 4b. Recall that the register allocation and instruction removal stages were merged. The majority of the stages show a scaling similar to the stages in Figure 4a, but a notable exception is the register allocation stage. As discussed in Section 3.5, lifetime analysis must be performed in parallel for each function, resulting in less parallelism being available to exploit, leading to a significant overhead compared to the other stages.

In Figure 5 the total runtime of the compiler is shown, excluding data transfer to/from the GPU. The total runtime is dominated by the register allocation stage. Still, a sub-linear scaling is seen, for instance from the fifth to seventh input file we observe an increase in runtime of 11.3x against a 44.5x increase in input size. This signifies that the parallelization of the different stages is effective.

A detailed performance comparison to other approaches is hard at this moment. On the one hand, given that we have presented the first compiler fully hosted on the GPU, there is no GPU baseline to compare to. A comparison to our compiler generated for the CPU would not yield a fair comparison as Futhark only generates multi-threaded CPU code, without using SIMD, causing not all available resources to be used. Moreover, all stages were designed with the high memory bandwidth available on GPUs in mind. On the other hand, a direct comparison to traditional CPU-based compilers would not be fair, as our compiler does not fully handle a programming language like C and does not implement all optimizations and instruction selection techniques. For a factual comparison the feature set of the GPU compiler must be up to par and also the quality of the produced code must be included in the comparison.

Still, we would like to obtain an initial performance comparison to see whether this development is worthwhile. To do so, we have translated the custom source files from the above experiments to C source files and measured the runtime of invoking `cc -w -c` on these files. The experiments were performed on an Intel Core i7-8700, using gcc 10.2.0 and Clang 8, and were repeated 10 times. The results are included as dashed lines in Figure 5. The C files are between 19.1% to 21.6% larger compared to the original files. As can be seen, the CPU-based compilers perform better for small input sizes, because of the overhead encountered by the GPU implementation. Once the input sizes get larger, the GPU implementation clearly shows better scaling compared to the CPU compilers. This shows that compilation on the GPU can be feasible with reasonable performance and that there is the potential to GPU-accelerate the compilation of large input files. Note that there is the potential to lower the bound on the input file size for GPU compilation to be profitable when register allocation is further optimized.

5 DISCUSSION AND FUTURE WORK

There are many avenues for further work. Most importantly, the identified performance bottlenecks need to be addressed to further increase the potential of GPU compilation. The large overhead of register allocation can possibly be alleviated by a redesign to operate on the level of expressions instead of functions. For the